

Aggregate Functions

Summarise and Analyse Data at Scale

LEARNING OUTCOME

minimums,

Skills You Will Gain

- Use COUNT, SUM, AVG, MIN, MAX correctly
- Understand how NULLs affect aggregate functions
- Use DISTINCT with aggregates
- Apply COUNT(*) vs COUNT(column) correctly
- Calculate percentages and ratios in SQL
- Use STRING_AGG and array aggregation functions

■ Aggregate functions are how data becomes insight. Every KPI dashboard, business report, and analytics query uses aggregates. COUNT, SUM, AVG, MIN, MAX are the five functions every data professional uses daily.

SECTION 1

Core Aggregate Functions

	What It Does	NULL Behaviour
	Count ALL rows including NULLs	Never ignores rows
	Count rows where col is NOT NULL	Ignores NULL values
	Total of all numeric values in col	Ignores NULLs
	Arithmetic mean of non-NULL values	Ignores NULLs -- affects result!
	Lowest value (works on text and dates)	Ignores NULLs
	Highest value (works on text and dates)	Ignores NULLs
	Standard deviation	Ignores NULLs
	Variance	Ignores NULLs

```
-- COUNT variations
SELECT COUNT(*) AS total_rows FROM customers;
SELECT COUNT(phone) AS has_phone FROM customers;
-- excludes NULLs
SELECT COUNT(DISTINCT city) AS unique_cities FROM customers;
-- count unique
SELECT COUNT(DISTINCT customer_id) AS buyers FROM orders;
-- distinct buyers
-- SUM
SELECT SUM(total_amount) AS total_revenue FROM orders;
SELECT SUM(quantity) AS units_sold FROM order_items;
SELECT SUM(total_amount) AS revenue, SUM(CASE WHEN status='cancelled' THEN total_amount ELSE 0 END) AS cancelled_revenue FROM orders;
-- AVG
SELECT AVG(total_amount) AS avg_order_value FROM orders;
SELECT AVG(annual_spend) AS avg_customer_spend FROM customers;
-- AVG ignores NULLs
-- if some rows are NULL, only non-NULL rows counted
-- To include NULLs as zero:
AVG(COALESCE(col, 0))
-- MIN and MAX
SELECT MIN(order_date) AS first_order, MAX(order_date) AS latest_order, MIN(total_amount) AS smallest_order, MAX(total_amount) AS largest_order FROM orders;
-- Multiple aggregates in one query
SELECT COUNT(*) AS total_orders, COUNT(DISTINCT customer_id) AS unique_customers, SUM(total_amount) AS total_revenue, AVG(total_amount) AS avg_order_value, MIN(total_amount) AS smallest_order, MAX(total_amount) AS largest_order, SUM(total_amount) / COUNT(DISTINCT customer_id) AS revenue_per_customer FROM orders WHERE status = 'delivered';
```

SECTION 2

Conditional Aggregation with CASE

```
-- COUNT with condition (pivot rows to columns)
SELECT COUNT(*) AS total_orders, COUNT(CASE WHEN status = 'delivered' THEN 1 END) AS delivered, COUNT(CASE WHEN status = 'cancelled' THEN 1 END) AS cancelled, COUNT(CASE WHEN status = 'pending' THEN 1 END) AS pending,
-- Percentage of cancelled orders
ROUND( 100.0 * COUNT(CASE WHEN status='cancelled' THEN 1 END) / COUNT(*), 2 ) AS cancel_rate_pct FROM orders;
-- SUM with condition (conditional total)
SELECT SUM(total_amount) AS total_revenue, SUM(CASE WHEN status = 'delivered' THEN total_amount ELSE 0 END) AS collected_revenue, SUM(CASE WHEN is_premium THEN annual_spend ELSE 0 END) AS premium_revenue, SUM(CASE WHEN MONTH(order_date) = 1 THEN total_amount ELSE 0 END) AS jan_revenue, SUM(CASE WHEN MONTH(order_date) = 2 THEN total_amount ELSE 0 END) AS feb_revenue, SUM(CASE WHEN MONTH(order_date) = 3 THEN total_amount ELSE 0 END) AS mar_revenue FROM orders o JOIN customers c ON o.customer_id = c.customer_id;
-- NULL trap with AVG
-- Wrong: if 10 customers have spend data, 5 have NULL
-- AVG(annual_spend) divides by 10, ignoring 5 nulls
-- Correct if NULL means zero spend:
SELECT AVG(COALESCE(annual_spend, 0)) AS true_avg_spend FROM customers;
```

SECTION 3

String and Statistical Aggregates

```
-- STRING_AGG (PostgreSQL) / GROUP_CONCAT (MySQL) -- concatenate values SELECT customer_id,
STRING_AGG(product_name, ' ' ORDER BY product_name) AS products_ordered -- PostgreSQL FROM orders
o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY customer_id; -- MySQL equivalent SELECT
customer_id, GROUP_CONCAT(product_name ORDER BY product_name SEPARATOR ' ') AS products FROM
orders o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY customer_id; -- Statistical
functions SELECT AVG(total_amount) AS mean, STDDEV(total_amount) AS std_deviation,
VARIANCE(total_amount) AS variance, -- Coefficient of variation STDDEV(total_amount) /
AVG(total_amount) * 100 AS cv_percent FROM orders; -- Percentile (PostgreSQL) SELECT
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total_amount) AS median, PERCENTILE_CONT(0.25) WITHIN
GROUP (ORDER BY total_amount) AS p25, PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY total_amount) AS
p75, PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY total_amount) AS p90 FROM orders;
```

■ PRACTICE Q & A

Q1. What is the difference between COUNT(*) and COUNT(column)?

A: COUNT(*) counts ALL rows including those with NULLs. COUNT(column) counts only rows where that column is NOT NULL. Use COUNT(*) for total rows, COUNT(col) when you need to count non-null entries.

Q2. How do NULLs affect AVG?

A: AVG ignores NULL values -- it only averages non-NULL rows. If 10 rows exist but 3 have NULL, AVG divides by 7 not 10. If NULLs should count as zero, use AVG(COALESCE(col, 0)).

Q3. What is conditional aggregation?

A: Using CASE WHEN inside an aggregate function to count or sum only rows meeting a condition. SUM(CASE WHEN status='delivered' THEN amount ELSE 0 END) gives revenue from delivered orders only.

Q4. What is the difference between SUM and COUNT?

A: SUM adds up the numeric VALUES in a column. COUNT counts the NUMBER of rows. SUM(quantity) gives total units; COUNT(quantity) gives how many rows have a quantity value.

Q5. What does COUNT(DISTINCT col) do?

A: Counts the number of unique distinct non-NULL values in a column. COUNT(DISTINCT customer_id) in orders gives the number of unique customers who placed orders.

■ Aggregate Functions Cheat Sheet	
COUNT(*)	Total rows (includes NULLs)
COUNT(col)	Rows where col is not NULL
COUNT(DISTINCT col)	Unique non-NULL values
SUM(col)	Total of numeric values
AVG(col)	Mean (ignores NULLs)
AVG(COALESCE(col, 0))	Mean treating NULL as zero
MIN(col) / MAX(col)	Lowest and highest value
STDDEV(col)	Standard deviation
SUM(CASE WHEN x THEN amt ELSE 0 END)	Conditional sum
COUNT(CASE WHEN x THEN 1 END)	Conditional count
STRING_AGG(col, ' ')	Concatenate values (PostgreSQL)
GROUP_CONCAT(col)	Concatenate values (MySQL)